

Software Engineering

Tahir Farooq
FAST-NU

Architectural Styles

Architectural Styles

- The architectural model of a system may conform to a **generic architectural** model or style
- An awareness of these styles can simplify the problem of defining system architectures

- Each style describes a system category that encompasses:
 - 1) A **set of components** (e.g. database, computational modules) that perform a function required by a system
 - 2) A **set of connectors** that enable “communication, coordination and cooperation” among components
 - 3) **Constraints** that define how components can be integrated to form the system

Categories of Architectural Styles

- ***Hierarchal Software Architecture***

- Layered

- ***Distributed Software Architecture***

- Client Server
- SOA

- ***Data Flow Software Architecture***

- Pipe n Filter
- Batch Sequential

- ***Event Based Software Architecture***

- ***Data Centered Software Architecture***

- Black board
- Shared Repository

- ***Interaction-Oriented Software Architectures***

- Model View Controller

- ***Component-Based Software Architecture***

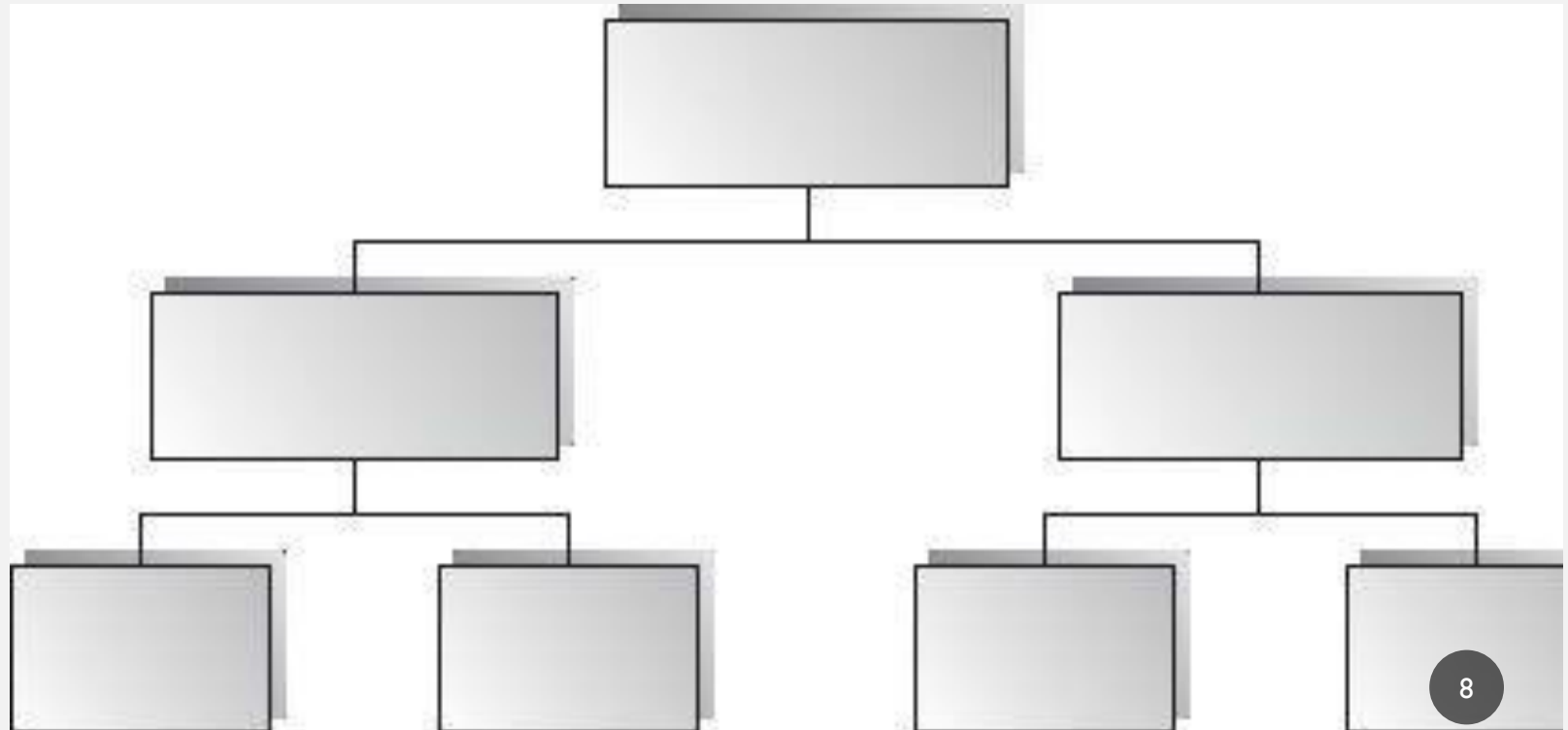
Hierarchal Software Architecture

Hierarchal Style

- The hierarchical software architecture is characterized by viewing the entire system as a hierarchy structure.
- The software system is **decomposed** into logical modules (subsystems) at **different levels** in the hierarchy.

Hierarchal Style

- Modules at different levels are connected by explicit or implicit method invocations.
- a lower-level module provides services to its adjacent upper-level modules, which invokes the methods or procedures in the lower level.



Hierarchal Style

- **System software** is typically designed using the hierarchical architecture style;
- examples include Microsoft .NET, Unix operating system, TCP/IP, etc.



Hierarchal Style

- **Lower levels** provide more specific functionality down to fundamental utility services
 - such as I/O services, transaction, scheduling, and security services, etc.
- **Middle layers**, in an application setting, provide more domain- dependent functions
 - such as business logic or core processing services.
- **Upper layers** provide more abstract functionality in the form of user interfaces
 - such as command line interpreters, GUIs, Shell programming facilities, etc.

Layered Architecture

Layered Style

- Organized hierarchically into layers.
- **Each layer provides service** to the layer **above** it and serves as a **client** to the layer **below**.
- The connectors are defined by the protocols that determine how the layers will interact.

A generic Layered Architecture

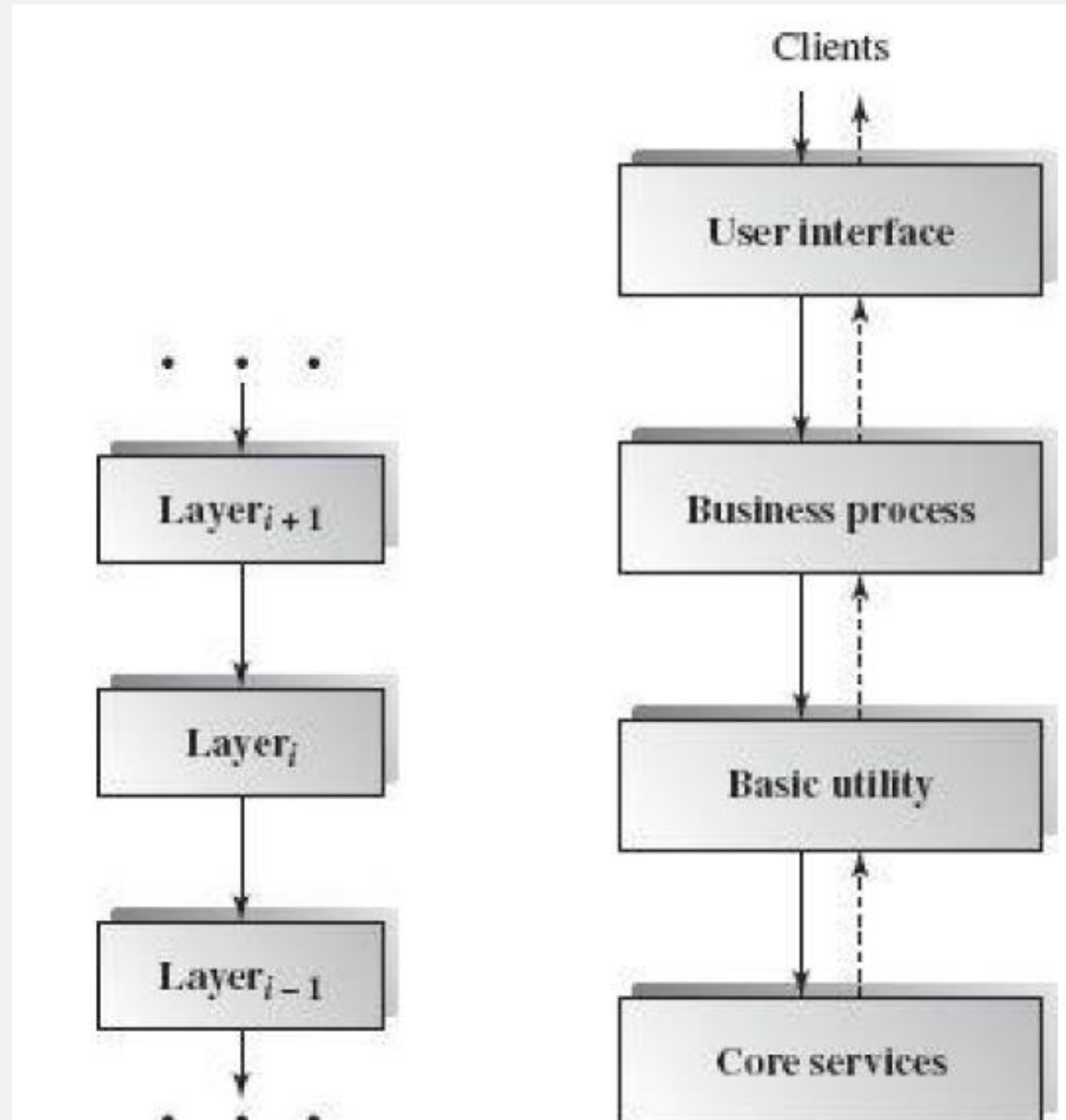
User Interface

User Interface Management
Authentication and Authorization

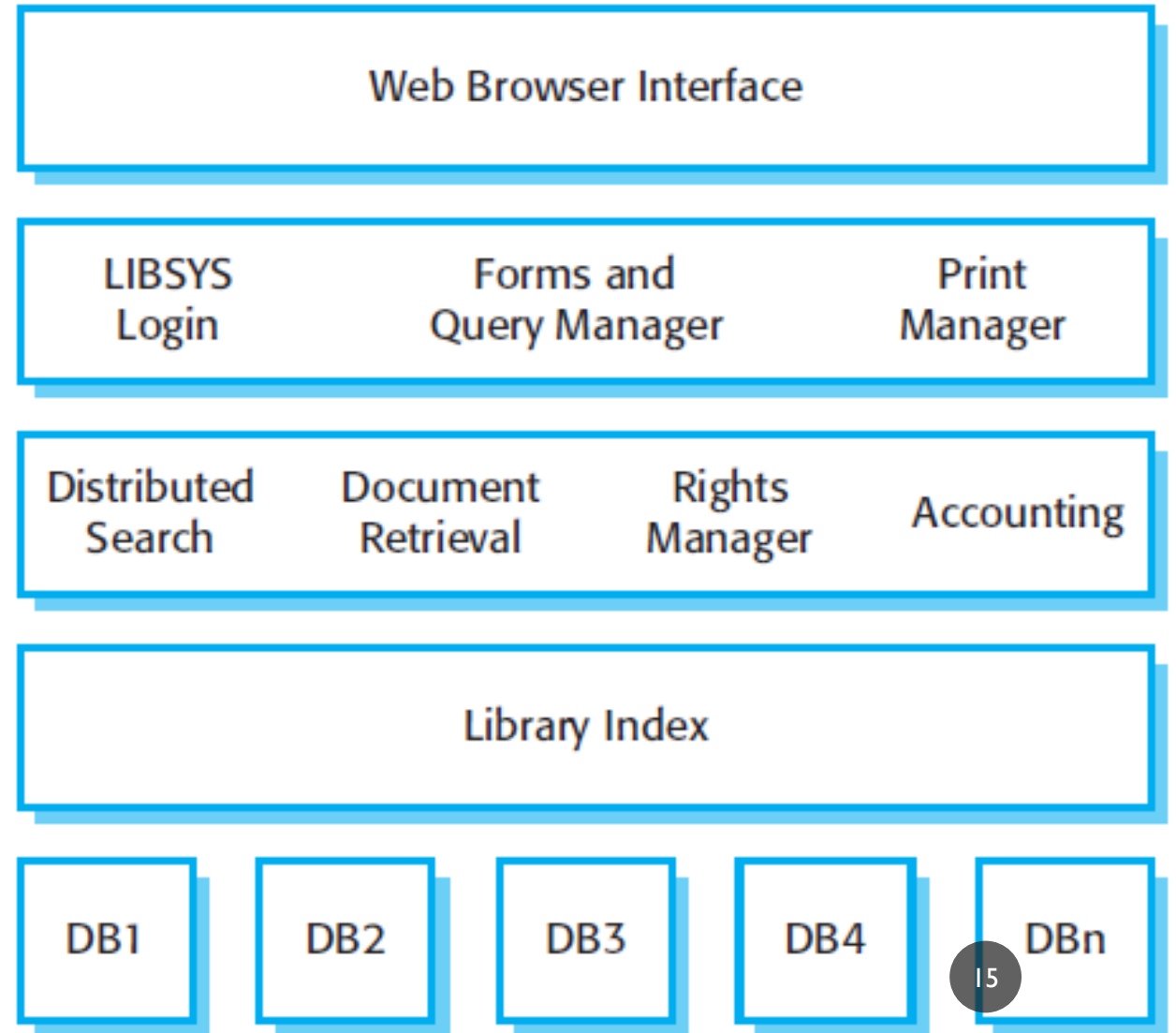
Core Business Logic/Application Functionality
System Utilities

System Support (OS, Database etc.)

A partial layered architecture example



Example: Library System



Ref: Software Engineering - Sommerville

Benefits:

- Incremental software development based on increasing levels of abstraction.
- Enhanced independence of upper layer to lower layer since there is no impact from the changes of lower layer services as long as their interfaces remain unchanged.

Benefits:

- Enhanced flexibility: interchangeability and reusability are enhanced due to the separation of the standard interface and its implementation.

Benefits:

- Component-based technology is a suitable technology to implement layered architecture; this makes it much easier for the system to allow for plug-and-play of new components.
- Promotion of portability: each layer can be an abstract machine deployed independently.

Limitations:

- Lower runtime performance since a client's request or a response to a client must go through potentially several layers.
- There are also performance concerns of overhead on the data processing and buffering by each layer.

- Many applications cannot fit this architecture design.
- Breach of interlayer communication may cause deadlocks, and “bridging” may cause tight coupling.
- Exceptions and error handling are issues in the layered architecture, since faults in one layer must propagate upward to all calling layers.

Distributed Software Architecture

Distributed Software Architecture

- A distributed system is a **collection** of **computational** and **storage devices** connected through a **communications network**.
- Data, software, and users are distributed.
- **Communication** occurs using a number of methods including **message passing**, **remote procedure calls**, and **remote method invocation**.

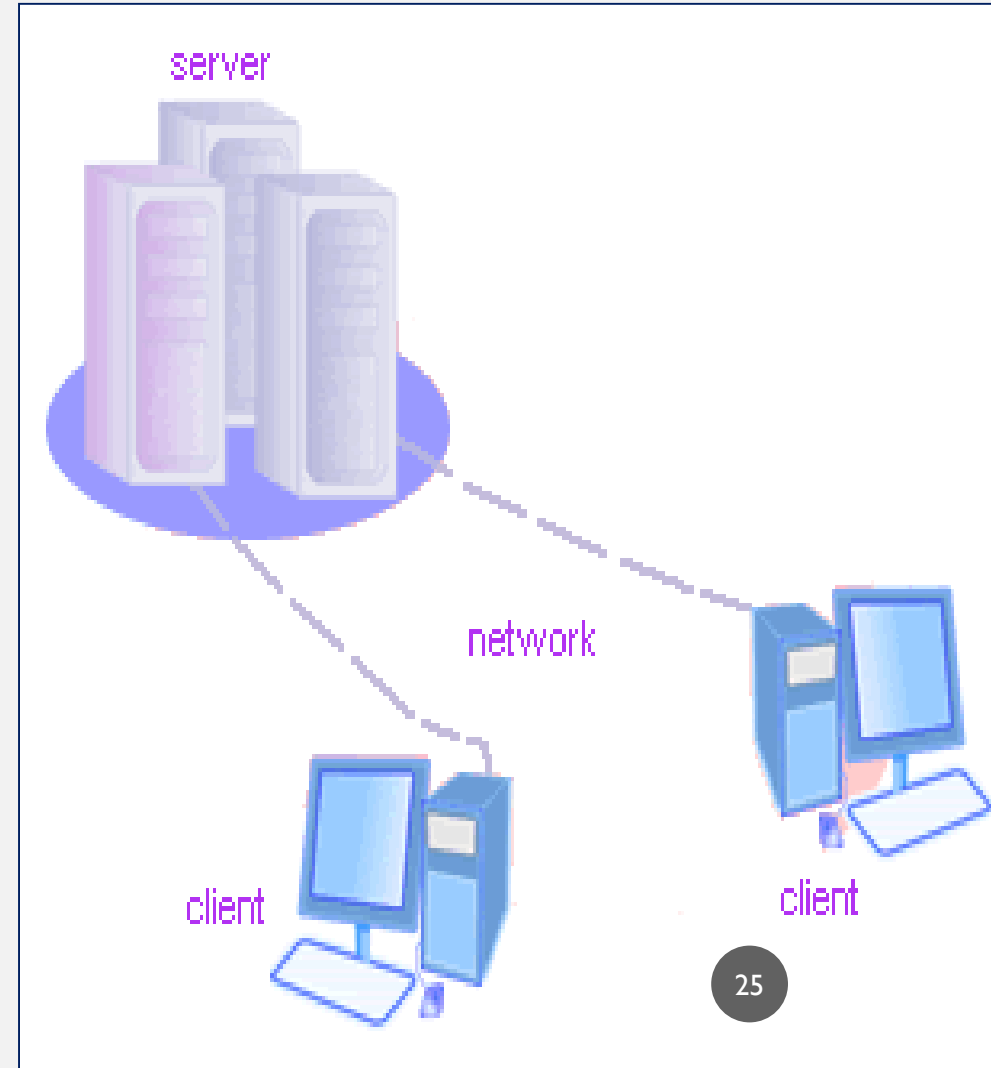
Client Server Architectural Style

Client Server Architectural Style

- Client/server architecture illustrates the **relationship between two computer** programs in which one program is a **client**, and the other is **Server**.
- **Client** makes a service request to server.
- **Server** provides service to the request.

Client/Server

- Although the client/server architecture can be used within a single computer by programs, but it is a more important idea in a network.
- In a network, the client/server architecture allows efficient way to interconnect programs that are distributed efficiently across different locations.

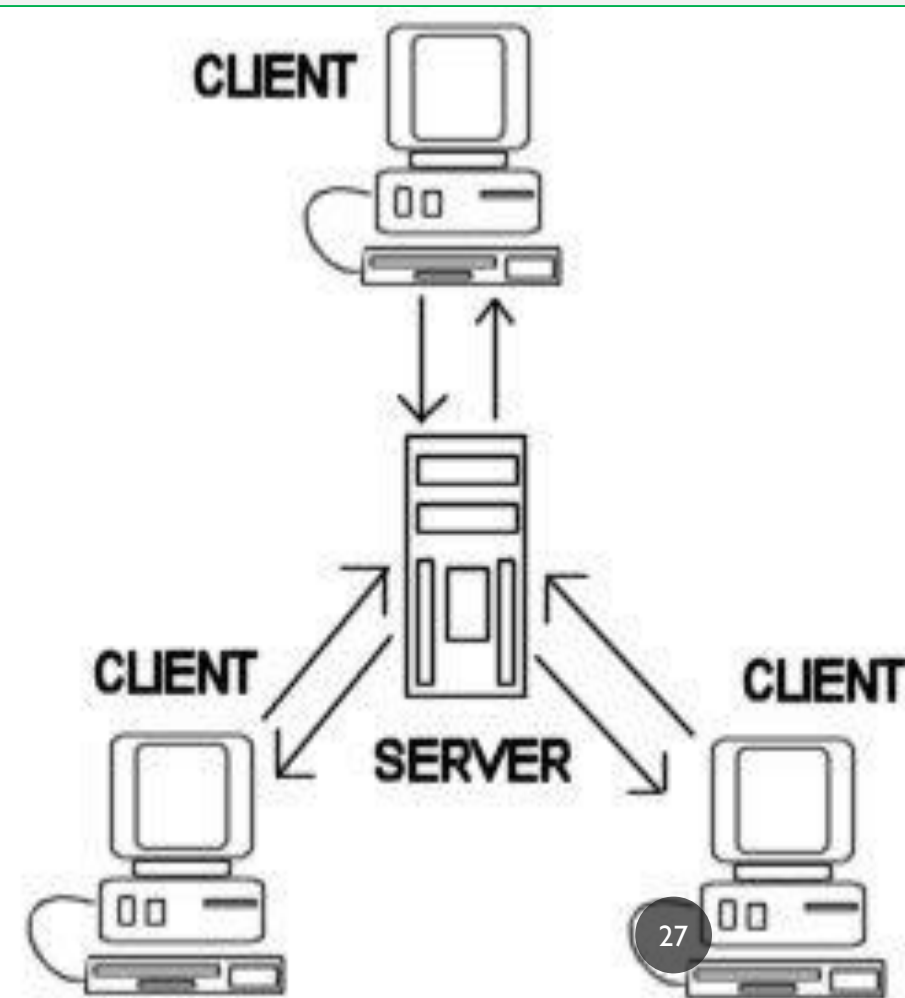


Client-server Style

- **Suitable** for applications that involve **distributed data** and processing across a range of components.
- **Components:**
 - **Servers:** Stand-alone components that provide specific services such as printing, data management, etc.
 - **Clients:** Components that call on the services provided by servers.
- **Connector:** The network, which allows clients to access remote servers.

Common Example

- The World Wide Web is an example of client-server architecture.
- Each computer that uses a Web browser is a client, and the data on the various Web pages that those clients access is stored on multiple servers.

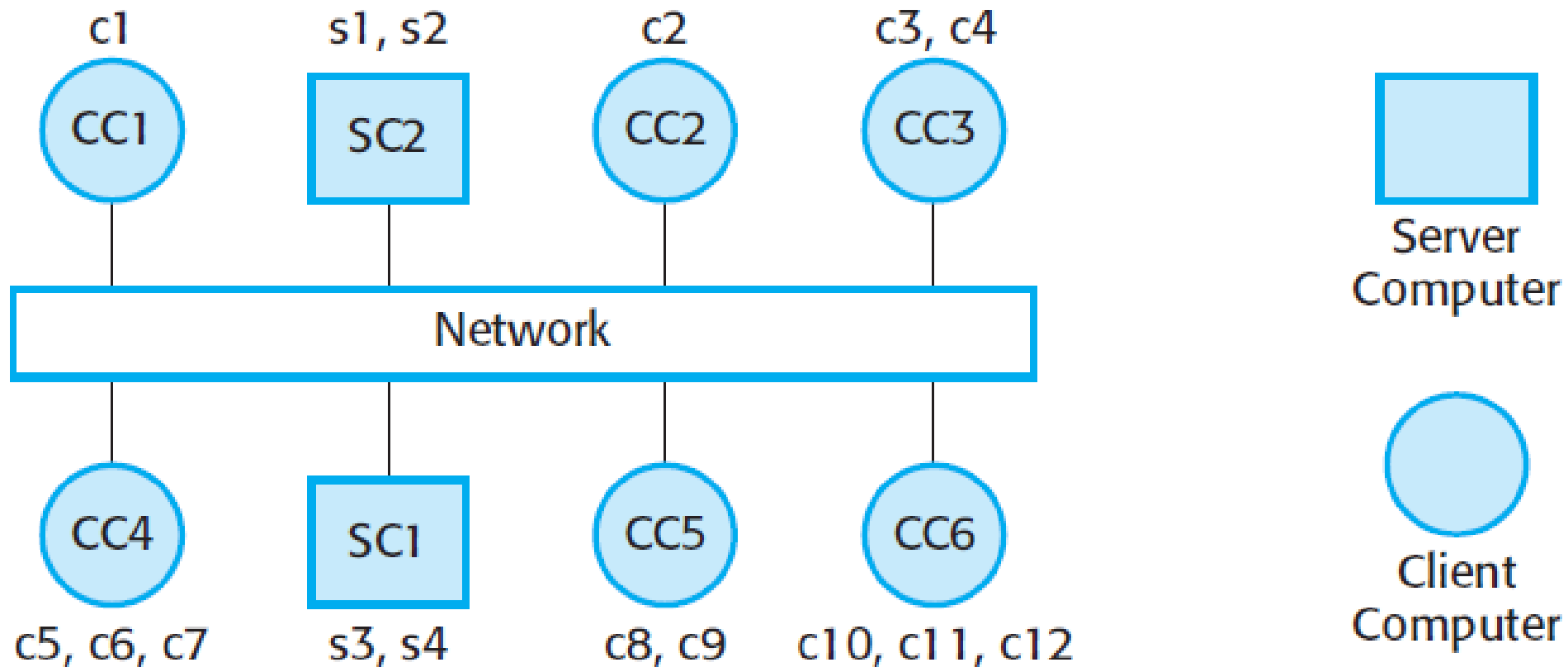


Another Example

- If you have to check a bank account from your computer, you have to send a request to a server program at the bank.
- That program processes the request and forwards the request to its own client program that sends a request to a database server at another bank computer to retrieve client balance information.
- The balance is sent back to the bank data client, which in turn serves it back to your personal computer, which displays the information of balance on your computer.

Client Server Processes

- Clients and servers are separate processes,



Client Server Processes

- It is normal for several client processes to run on a single processor.
 - For example, on your PC, you may run a mail client that downloads mail from a remote mail server.
 - You may also run a web browser that interacts with a remote web server and a print client that sends documents to a remote printer.

Client Server Processes

- Several different server processes may run on the same processor but, often,
 - servers are implemented as multiprocessor systems in which a separate instance of the server process runs on each machine.

Example:

- a) Gmail
- b) Picassa photoviewer
- c) Drop box

Severs

- Servers commonly contain data files and applications that can be accessed across the network, by workstations or user computers.
- A user who wants to access data files, would use his or her client computer to access the data files on the server.

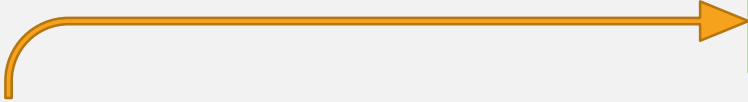
Types Of Servers

- **Application Servers:**
 - Applications are hosted on these servers
 - Clients can access various application features over the network

Cricinfo, results from university website

Types Of Servers

- **File Servers:**



Those file which We create with other software

- Primitive form of data service.
- Useful for sharing files across a network.
- The client passes requests for files over the network to the file server.

**Facebook save ours pictures,videos,
Different servers save our complete files**

Types Of Servers

- **Database Servers:**
 - More efficient use of distributing power than file servers.
 - Client passes SQL requests as messages to the DB server; results are returned over the network to the client.
 - Query processing done by the server.
 - No need for large data transfers.

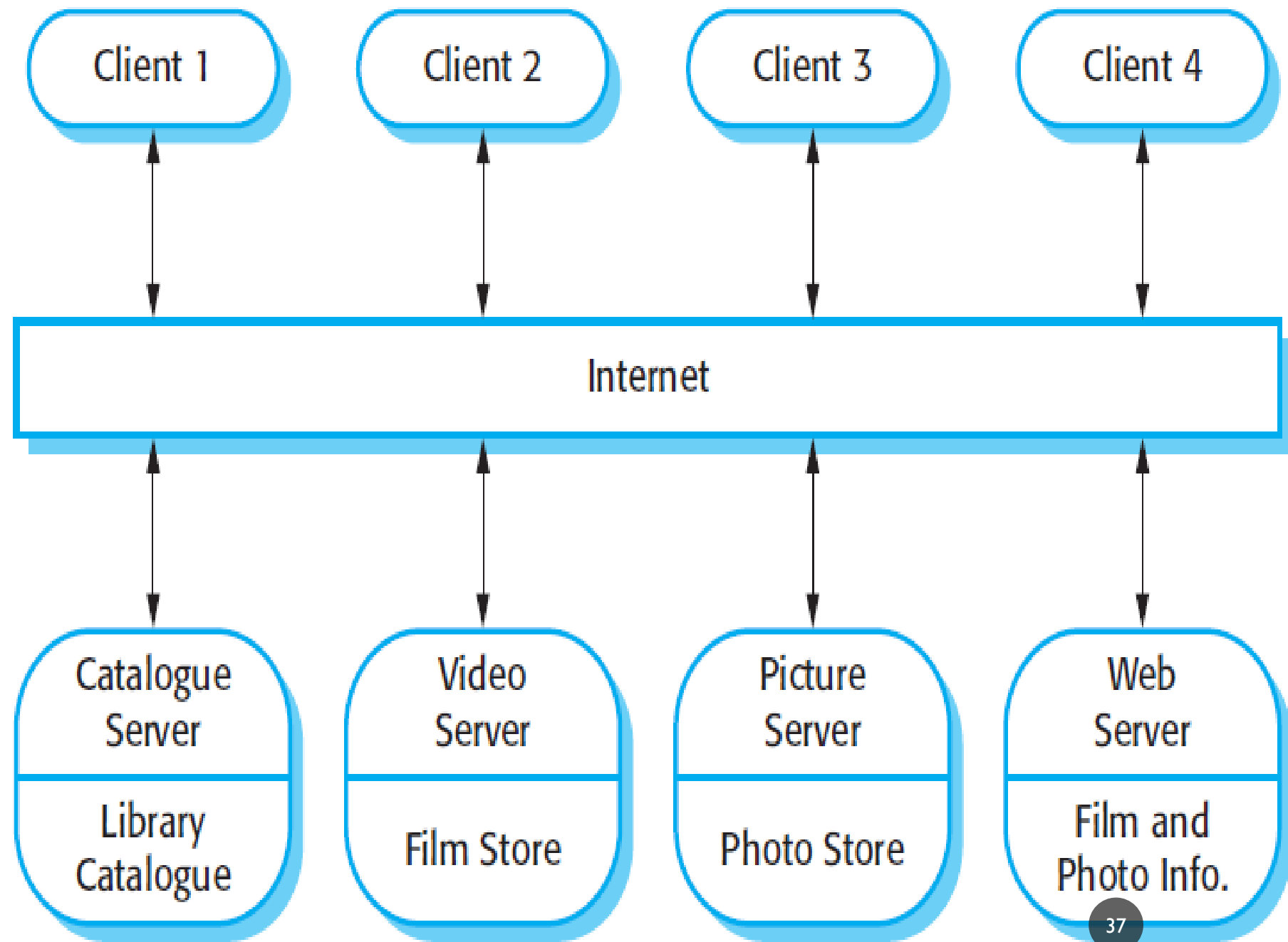


Figure 6.11 A client–server architecture for a film library

Advantages

- Straightforward distribution of data.
- Transparency of location.
- Mix and match heterogeneous platforms
- Easy to add new servers or upgrade existing servers.

Disadvantages

- Performance of the system depends on the performance of the network.
- Tricky to design and implement C/S systems.
- Unless there is a central register of names and services, it may be hard to find out what services are available.

Multi-tier Client Server Architecture

Types Of Client Server

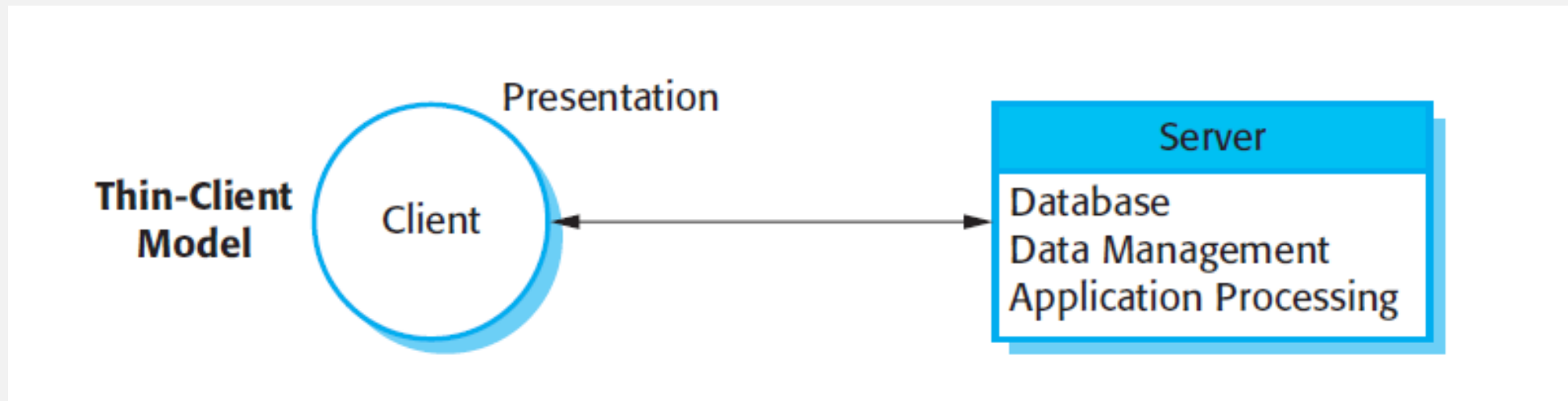
- **Two-tier client–server architecture**
 - which is used for simple client–server systems, and in situations where it is important to centralize the system for security reasons.
 - In such cases, **communication** between the client and server is normally **encrypted**.
- **Multitier client–server architecture**
 - which is used when there is a **high volume of transactions** to be processed by the **server**.

A Two-tier Client–server Architecture

- The system is implemented as a single logical server plus an indefinite number of clients that use that server.
- Two forms of this architectural model:
 - A thin-client model,
 - A fat-client model,

Two-tier Client Server

- A thin-client model,
 - where the **presentation layer** is implemented on the client and all other layers (data management, application processing, and database) are implemented on a server.



Advantages

- The advantage of the thin-client model is that it is simple to manage the clients.
- This is a major issue if there are a large number of clients, as it may be difficult and expensive to install new software on all of them. If a web browser is used as the client, there is no need to install any software.

Disadvantages

- The disadvantage of the thin-client approach, however is that it may place a heavy processing load on both the server and the network.
- The **server is responsible for all computation** and this may lead to the generation of significant network traffic between the client and the server.

Two-tier Client Server

- A **fat-client** model
- where **Some** or all of the application processing is carried out on the **client**.
- **Data management** and **database** functions are implemented on the **server**.

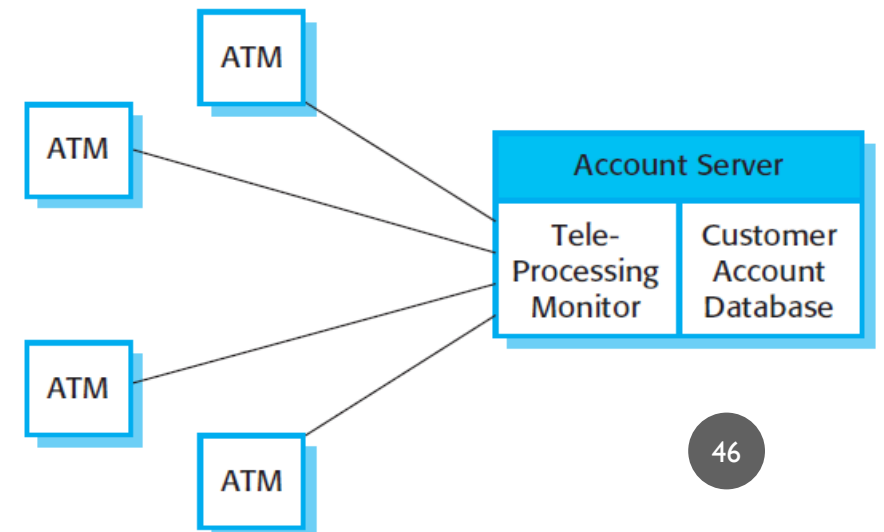
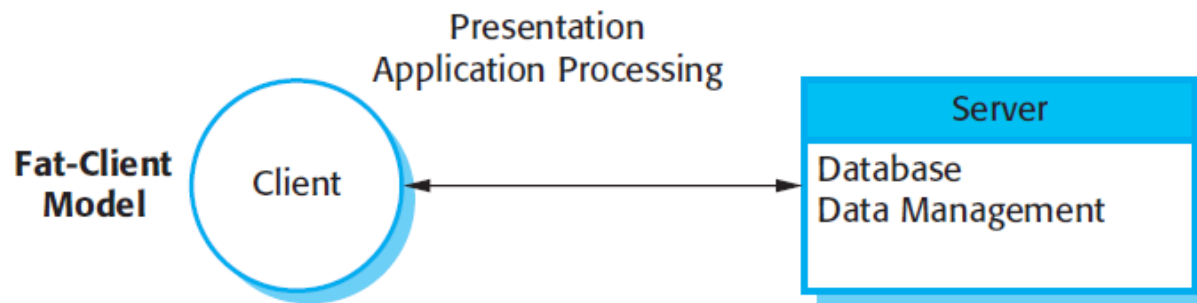
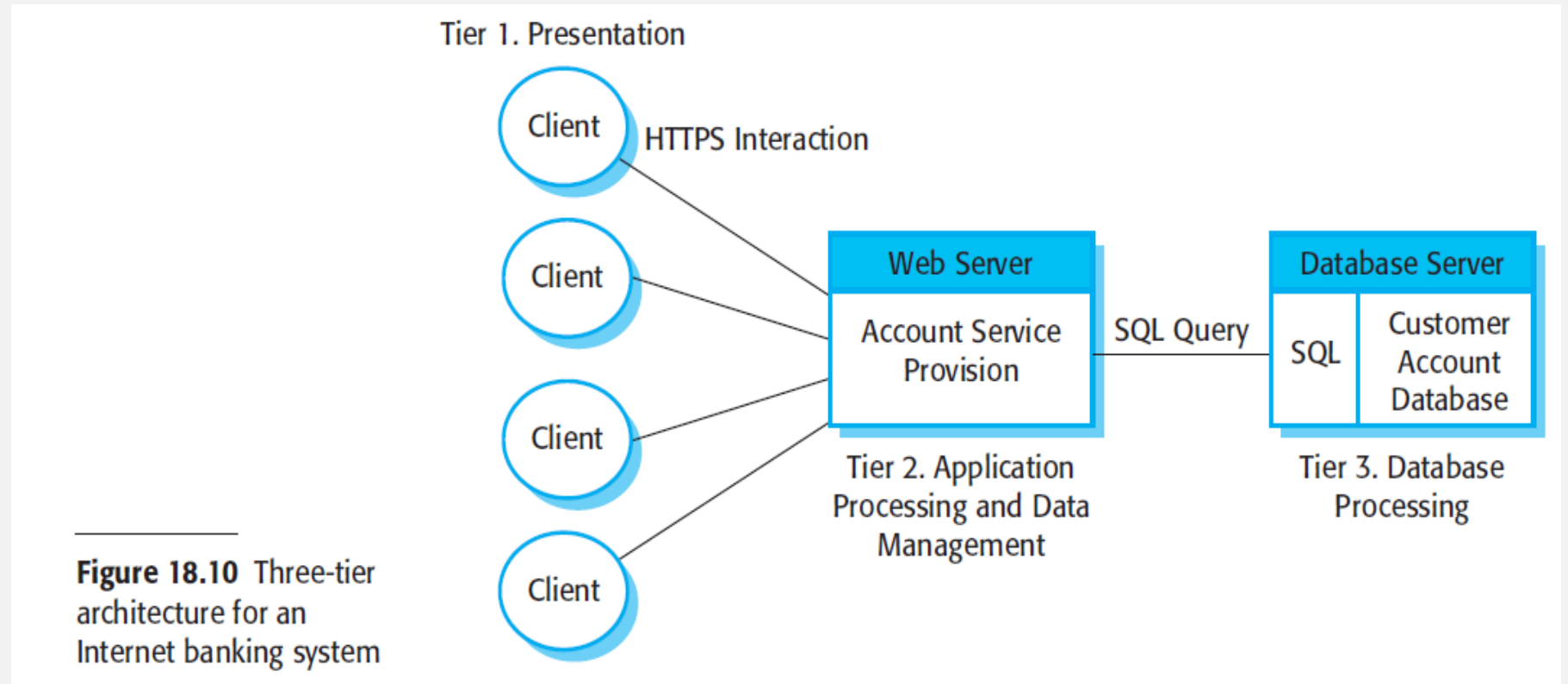


Figure 18.9 A fat-client architecture for an ATM system

Multi-tier Client–server Architectures

- The **fundamental problem** with a two-tier client–server approach is that the logical layers in the system—presentation, application processing, data management, and database—must be mapped onto two computer systems: the client and the server.
- This may lead to problems with **scalability** and **performance** if the **thin-client model** is chosen, or problems of **system management** if the **fat-client model** is used.

- Application services such as facilities to transfer cash, generate statements, pay bills, and so on are implemented in the web server and as scripts that are executed by the client



- This system is scalable because it is relatively easy to add servers (scale out) as the number of customers increase.
- In this case, the use of a three-tier architecture allows the information transfer between the web server and the database server to be optimized.

Issues

Security for each server

Banking System-> need security but they have to handle their maximum users also

Shared Data Software Architecture

Shared Data Software Architecture

- Data-centered software architecture is characterized by a centralized data store that is shared by all surrounding software components.
- The software system is decomposed into two major partitions: data store and independent software component or agents.

Shared Data Software Architecture

- The connections between the data module and the software components are implemented either by **explicit method invocation** or by **implicit method invocation**.

Shared Data Software Architecture

- In pure data-centered software architecture, the software components **don't communicate** with each other directly; instead, all the communication is conducted via the **data store**.
- The **shared data module** provides all mechanisms for software components to access it, such as **insertion, deletion, update, and retrieval**.

Shared Data:

- The high level design solution is based on a shared data-store which acts as the “central command” with 2 variations:
 - Blackboard style
 - Repository style

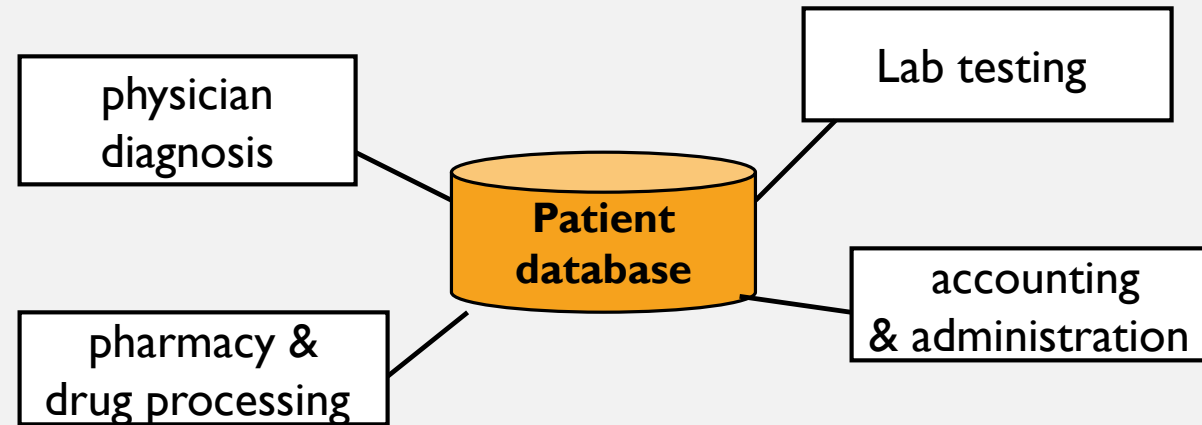
REPOSITORY ARCHITECTURE

Repository Architecture

- The repository architecture style is a **data-centered** architecture that **supports user interaction** for data processing.
- The software component agents of the data store control the computation and flow of logic of the system.

Repository architecture

- All data in a system is managed in a central repository that is accessible to all system components.
- Components do not interact directly, only through the repository.



Repository architecture

- Organizing tools around a repository is an efficient way to share large amounts of data.
- There is no need to transmit data explicitly from one component to another.

Repository architecture

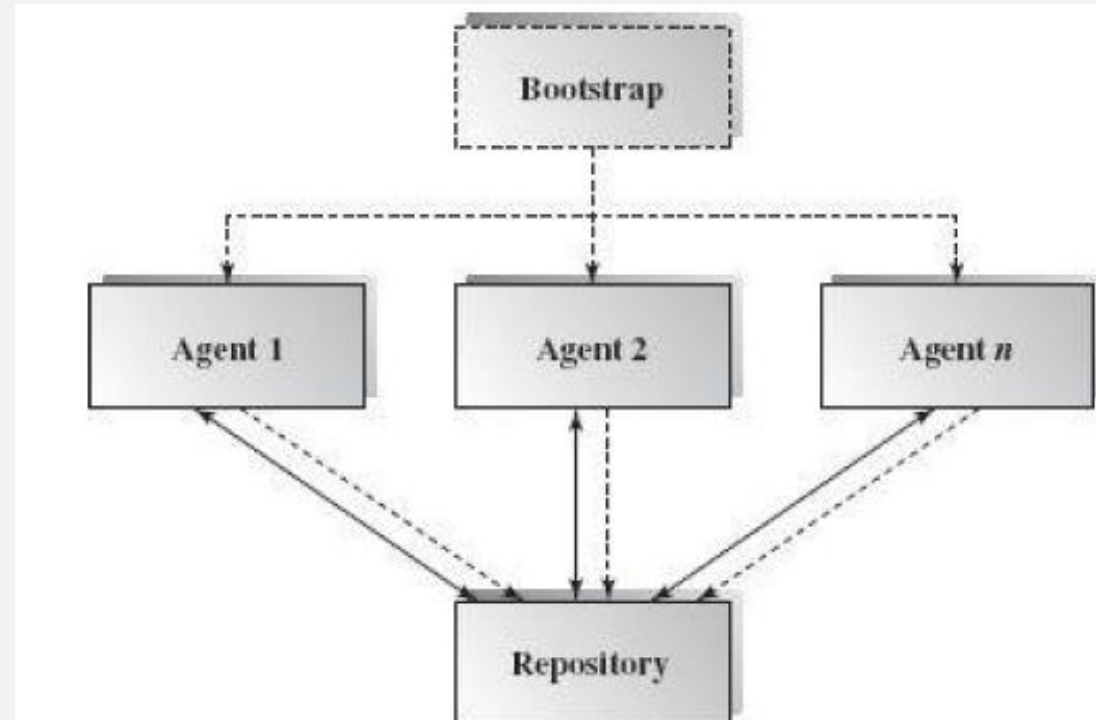
- However, components must operate around an **agreed repository data model**.
- **Inevitably**, this is a compromise between the specific needs of each tool and it may be difficult or impossible to integrate new components if their data models do not fit the agreed schema.

Repository architecture

- Although it is possible to distribute a logically centralized repository, there may be problems with data redundancy and inconsistency.
 - In practice, it may be difficult to distribute the repository over a number of machines.

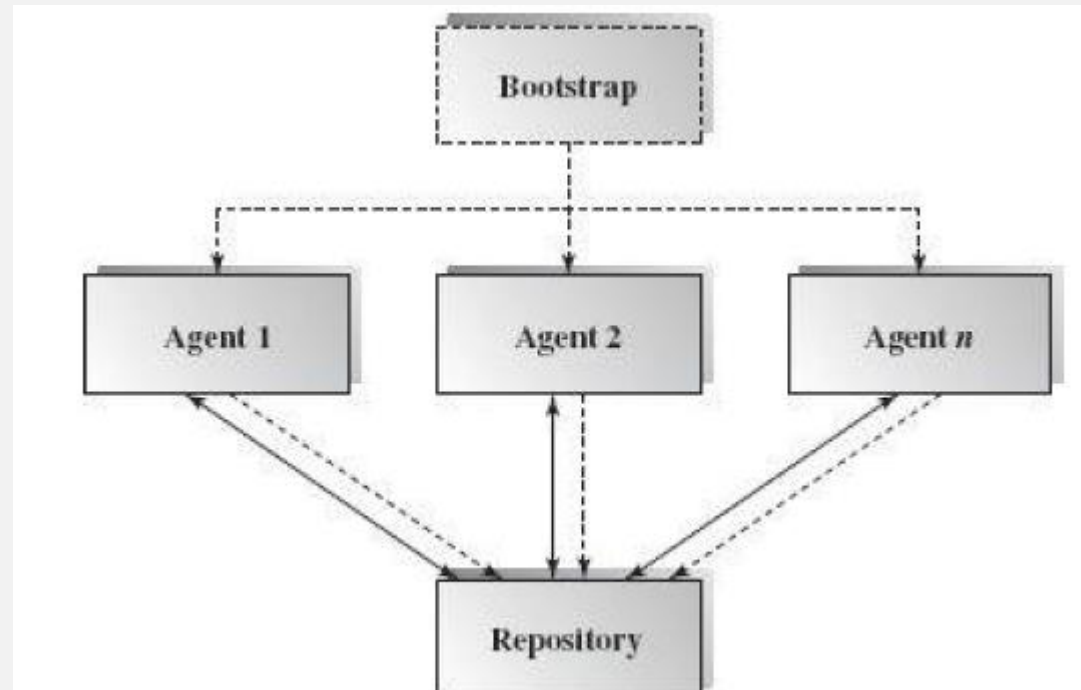
Repository Architecture

- A general picture of the repository architecture.
- The dashed lines pointing toward repository indicate that repository clients have full control over the logic flow.



Repository Architecture

- Clients can get data from the data store and put data in the data store.
- Different clients may have different interfaces and different data access privileges.



Example: Integrated Development Environment

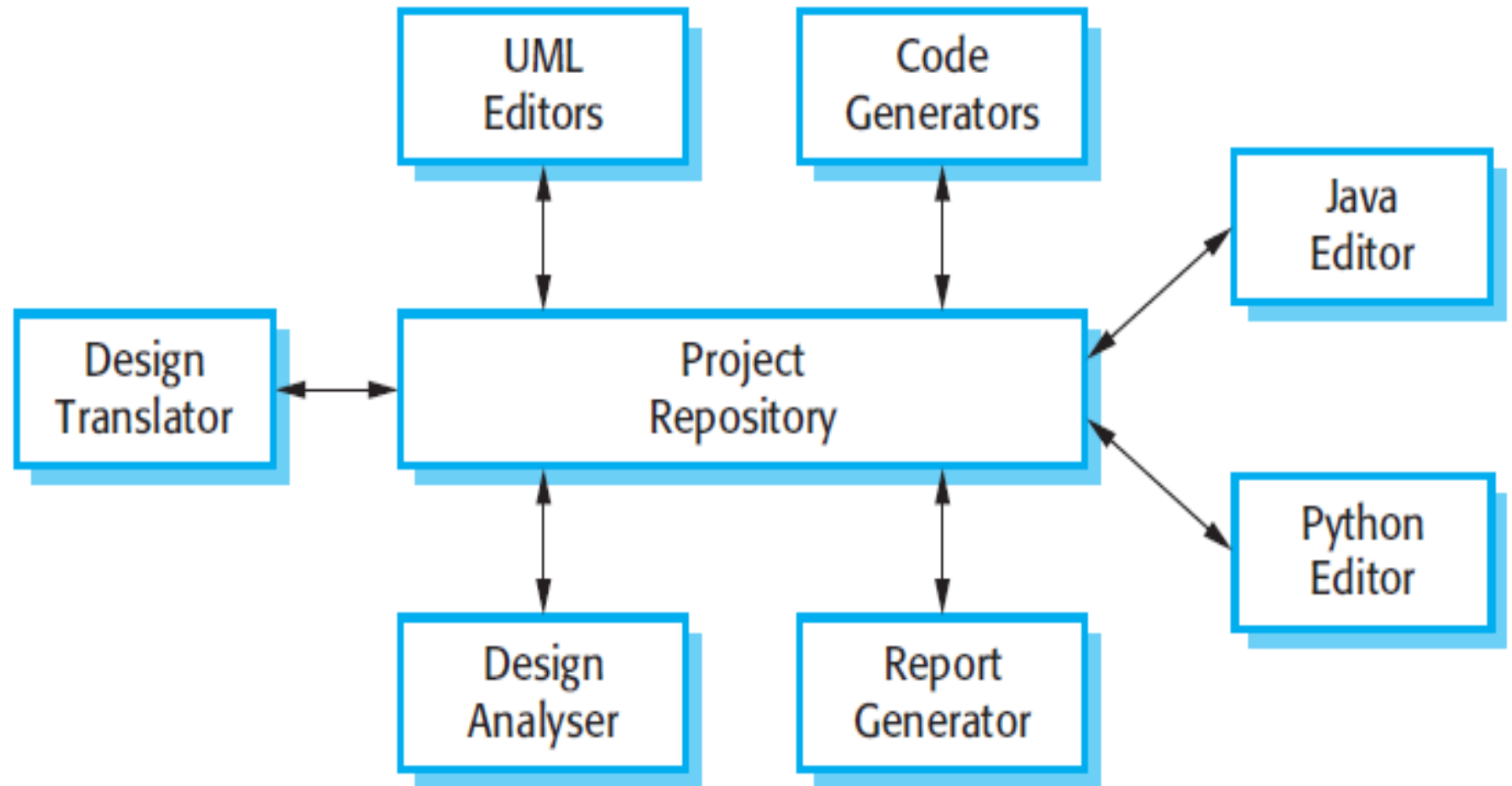
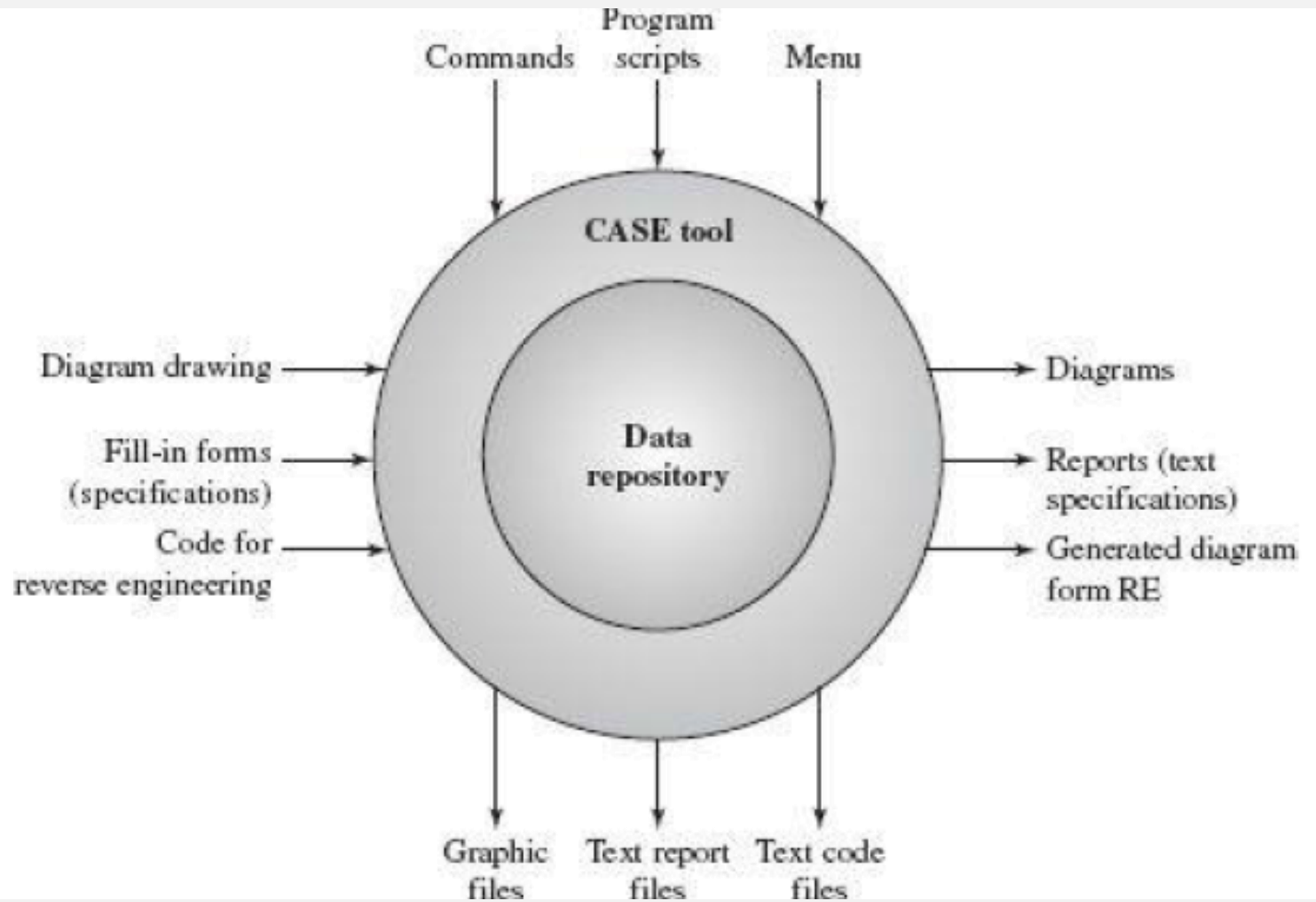


Figure 6.9 A repository architecture for an IDE

CASE Tools

- There are many CASE tools surrounding the data store



CASE tools

- A user of CASE tools can draw a UML design diagram such as a class diagram, collaboration diagram, or sequence diagram, and store the design blueprints in the CASE data store.
- The biggest advantage of CASE tools is its centralized data with many supporting software tools which can generate different products for different purposes based on the same set of data.

Applicable domains of repository architecture:

- Suitable for large, complex information systems where many software component clients need to access them in different ways
- Requires data transactions to drive the control flow of computation

Advantages

- Efficient way to share large amount of data
- Sub-systems need not be concerns with how data is produced (Centralized management). E.g. backup, security, etc.
- Sharing model is published as the repository schema.

Advantages

- Components can be independent—they do not need to know of the existence of other components.
- Changes made by one component can be propagated to all components.
- All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.

Disadvantages

- The repository is a single point of failure so problems in the repository affect the whole system.
- Data evolution is difficult and expensive.
- May be inefficiencies in organizing all communication through the repository.
- Distributing the repository across several computers may be difficult.

Data Flow Software Architecture

Data Flow Architectures

- The data flow software architecture style **views the entire software system** as a series of **transformations on successive** sets of data, where data and operations on it are independent of each other.
- The software system is decomposed into data processing elements where data directs and controls the order of data computation processing.

Data Flow Architectures

- **Each component** in this architecture **transforms its input data** into corresponding **output data**.
- The connection between the subsystem components may be implemented as I/O streams, I/O files, buffers, piped streams, or other types of connections.

Data Flow Architectures

- Regardless of the type of topology, the data moves from one subsystem to another.
- In general, there is no interaction between the modules except for the output and the input data connections between subsystems.

Data Flow Architectures

- One subsystem can be substituted by another without affecting the rest of the system
- Since each subsystem does not need to know the identity of any other subsystem, modifiability and reusability are important property attributes of the data flow architecture.

Pipes-and-Filters Style

Pipe and Filter

- This architecture decomposes the whole system into components of data source, filters, pipes, and data sinks.
- The connections between components are data streams.
- The particular property attribute of the pipe and filter architecture is its concurrent and incremented execution.

Pipes-and-Filters Style

- Suitable for applications that require a defined series of independent computations to be performed on data.
- Each component has a set of inputs and set of outputs.
- A component reads a stream of data on its input and produces stream of data at its output.

Filter

- Each filter is an independent **data** stream transformer;
 - it reads data from its input data stream, transforms and processes it, and then
 - writes the transformed data stream over a pipe for the next filter to process.

Filter

- A filter does not need to wait for batched data as a whole.
 - As soon as the data arrives through the connected pipe, the filter can start working right away.
- A filter does not even know the identity of data upstream or data downstream.
 - A filter is just working in a local incremental mode.

Active and Passive filters

- An active filter pulls in data and pushes out the transformed data (pull/push);
- A passive filter lets connected pipes push data in and pull data out.

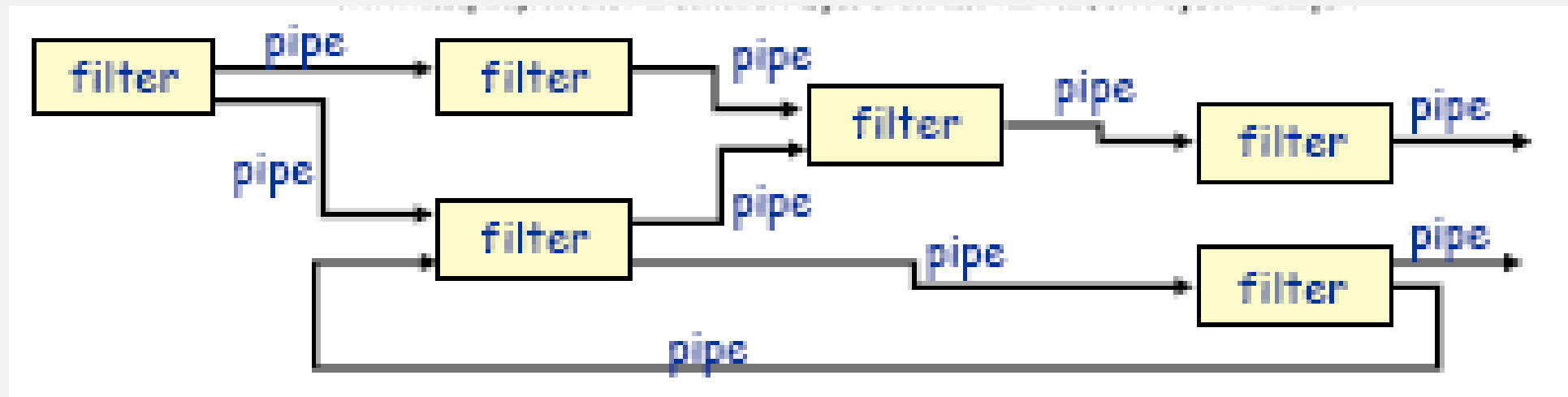
Pipes

- The connectors serve as channels for the streams, transmitting outputs of one filter to inputs of the other.
 - This makes connectors act as **Pipes**.

Pipes

- A pipe moves a data stream from one filter to another.
- A pipe can carry binary or character streams.
- A pipe is placed between two filters; these filters can run in separate threads of the same process.

Structure



There are three ways to make the data flow:

- Push only (Write only)
 - A data source may push data in a downstream.
 - A filter may push data in a downstream.
- Pull only (Read only)
 - A data sink may pull data from an upstream.
 - A filter may pull data from an upstream.
- Pull/Push (Read/Write)
 - A filter may pull data from an upstream and push transformed data in a downstream.

Examples

- **Traditional Compilers:**
- Compilation phases are pipelined, though the phases are not always incremental. The phases in the pipeline include:
 - *lexical analysis + syntax analysis (parsing) + semantic analysis + code optimization + code generation*

Example: Architecture of a Compiler

- Compilation is regarded as a sequential (pipeline) process.
- Every phase is dependent on some data on the preceding phase.



Compiler

- The system might consist of a pipeline of processing elements similar to the following:
 - A ***source*** element reads the program text (i.e., source code) from a file (or perhaps a sequence of files) as a stream of characters.
 - A ***lexical analyzer*** converts the stream of characters into a stream of lexical tokens for the language--keywords, identifier symbols, operator symbols, etc.
 - A ***parser*** recognizes a sequence of tokens that conforms to the language grammar and translates the sequence to an abstract syntax tree.

Compiler

- A "*semantic analyzer*" reads the abstract syntax tree and writes an appropriately augmented abstract syntax tree.
- A *global optimizer* (usually optionally invoked) reads an augmented syntax tree and outputs one that is equivalent but corresponds to program that is more efficient in space and time resource usage.
- An *intermediate code generator* translates the augmented syntax tree to a sequence of instructions for a virtual machine.

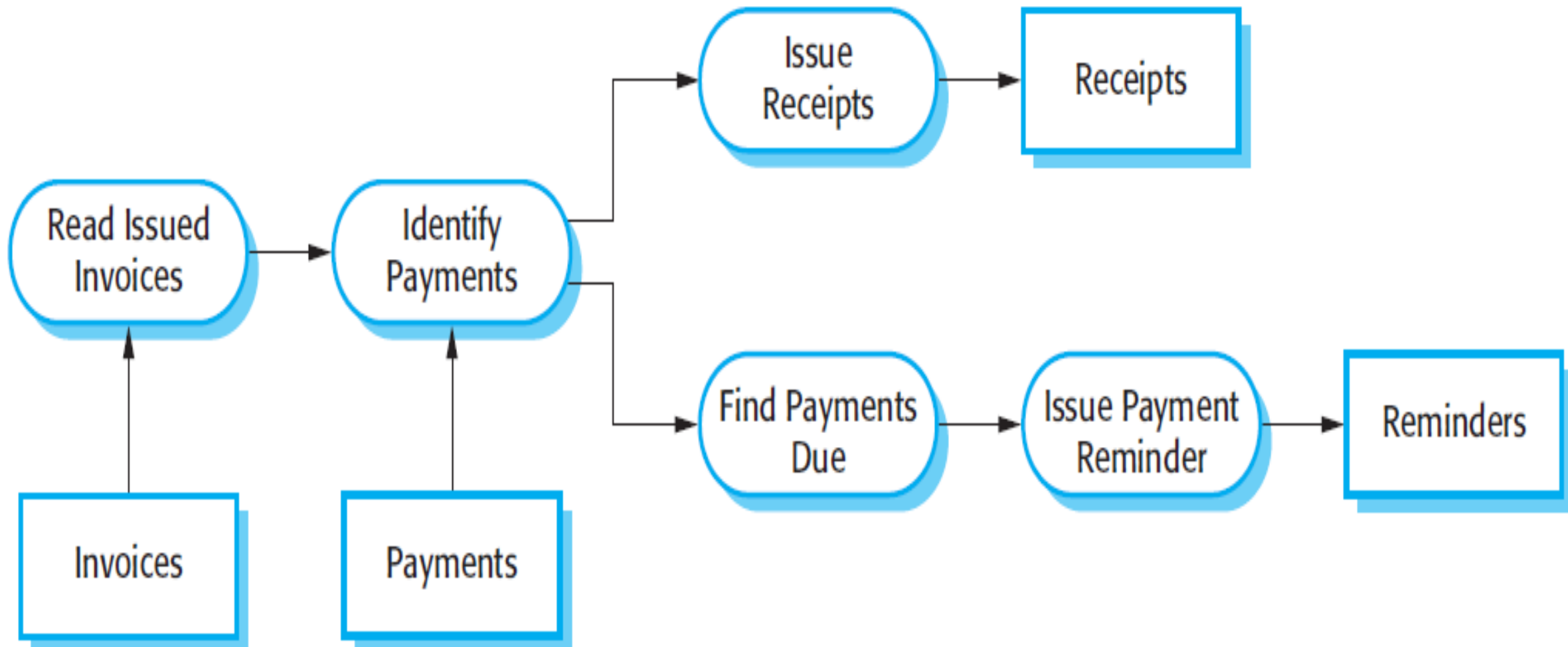
Compiler

- A *local optimizer* converts the sequence of intermediate code (i.e., virtual machine) instructions into a more efficient sequence.
- A *backend code generator* translates the sequence of virtual machine instructions into a sequence of instructions for some real machine platform (i.e., for some particular hardware processor augmented by operating system calls and a runtime library).

Compiler

- If the previous step generated symbolic assembly code, then an ***assembler*** is needed to translate the sequence of symbolic instructions into a relocatable binary module.
- If the previous steps of the pipeline generated a sequence of separate binary modules, then a ***linker*** might be needed to bind the separate modules with library modules to form a single executable (i.e., object code) module.
- A ***sink*** element outputs the resulting binary module into a file.

Sales System



Applicable domains of pipe and filter architecture:

- The system can be broken into a series of processing steps over data streams, and at each step filters consume and move data incrementally.
- The data format on the data streams is simple, stable, and adaptable if necessary.
- Significant work can be pipelined to gain increased performance.
- Producer or consumer-related problems are being addressed.

Benefits:

- Concurrency: It provides high overall throughput for excessive data processing.
- Reusability: Encapsulation of filters makes it easy to plug and play, and to substitute.
- Flexibility: It supports both sequential and parallel execution.

Benefits:

- **Modifiability:** It features low coupling between filters, less impact from adding new filters, and modifying the implementation of any existing filters as long as the I/O interfaces are unchanged.
- **Simplicity:** It offers clear division between any two filters connected by a pipe.

Disadvantages

- Not good choice for **interactive systems**, because of their transformational characteristic.